WEBCOEVO: Adversarial Co-Evolution for Web-Agent Reflection Rules Capability-Targeted Website Drift on LINKDING

Chenming Ge, Chengyang Shi, Yifei Xu, Yuxiang Yang, Binglin Zhong
University of Michigan
{gecm, chengysh, xuyifei, yyxxyj, tozenith}@umich.edu

Abstract

Web agents often fail when a website changes in ways that preserve the user's goal but disrupt learned interface shortcuts. We present WEBCOEVO, a single-site but end-to-end auditable loop for generating capability-targeted website drift and testing compact reflection rules. The loop profiles failures, builds fresh LINKDING website overlays, mines matched trajectory transitions, edits a small reflection rulebook, and evaluates the resulting policy with prompt-injection evidence recorded in trace JSONL. The upgraded WebCoEvo evidence covers three website versions: LINKDING V1.45.0, a moderately modified website V2, and a harder generated website suite V3 over seven drift categories. On the 68 training tasks from LINKDING V1.45.0, EXPEL-style rules raise Qwen3-VL success from $9/68 = 13.2\%$ to $56/68 = 82.4\%$. Under website V2, R1 reflection rules improve over EXPEL-style prompting from $60/68 = 88.2\%$ to $67/68 = 98.5\%$ on the training tasks and from $114/167 = 68.3\%$ to $143/167 = 85.6\%$ on the held-out validation tasks. On website V3, EXPEL-style prompting collapses to $8/68 = 11.8\%$ on the training tasks and $66/167 = 39.5\%$ on the held-out validation tasks, while R1 reaches $65/68 = 95.6\%$ and $97/167 = 58.1\%$. A GPT-5.4-assisted R4 update ties R1 on the held-out validation tasks but does not improve the training tasks, showing that iterative reflection is useful but not monotonic. We claim auditable LINKDING robustness improvement from compact reflection rules, not universal web-agent robustness.

1 Introduction

Web-agent benchmarks are becoming more realistic, but most still treat website change as a fixed nuisance: choose a set of versions or perturbations, run an agent, and report success rate. That protocol measures brittleness, yet it does not answer the design question that appears after each evaluation. Once an agent has a strong rulebook, which website changes still expose its capability boundary, and how should the next rule update be selected?

This question matters for reflection-based agents. A reflection rule such as "preserve a login redirect" or "stop when the target state is already visible" only helps if the benchmark contains valid situations where that behavior is required. If generated websites are too easy, they provide no learning signal. If they are broken, failures cannot support a rule. WEBCOEVO studies the middle ground: use the current agent as a solver, generate websites that target its observed frontier, and mine small cross-version reflection rules from paired trajectory evidence.

We instantiate this loop on LINKDING, a self-hosted bookmark manager with deterministic local deployment and code-level website overlays. The agent is Qwen3-VL in a BrowserGym/AgentLab/UI-TARS stack. The base policy can receive EXPEL task-experience rules distilled from earlier trajectories, while the cross-version layer injects compact reflection rules about how to adapt when the website changes. This separation is important. EXPEL rules tell the agent what helped on related tasks; cross-version reflection rules tell it how to preserve task intent across a shifted interface.

The upgraded WebCoEvo evidence has three website versions. The first is LINKDING V1.45.0, the original release. The second is a moderately modified website V2 that tests whether R1 reflection rules improve over EXPEL-style prompting under moderate website drift. The third is a harder generated website suite V3 with seven drift categories: `access`, `surface`, `content`, `runtime`, `process`, `structural`, and `functional`. Website V3 is the primary stress test because EXPEL-style prompting collapses there even when it performs well on LINKDING V1.45.0 and website V2.

The main empirical result is sharper than in the earlier draft. On the 68 training tasks from LINKDING V1.45.0, EXPEL-style rules raise success from $9/68 = 13.2\%$ to $56/68 = 82.4\%$. On website V2, R1 reflection rules improve over EXPEL-style prompting from $60/68 = 88.2\%$ to $67/68 = 98.5\%$ on the training tasks and from $114/167 = 68.3\%$ to $143/167 = 85.6\%$ on the held-out validation tasks. On website V3, however, EXPEL-style prompting drops to $8/68 = 11.8\%$ on the training tasks and $66/167 = 39.5\%$ on the held-out validation tasks. R1 recovers most of the training split, reaching $65/68 = 95.6\%$, and transfers to the held-out validation tasks at $97/167 = 58.1\%$. R4, a GPT-5.4-assisted update built from transition evidence, ties R1 on the held-out validation tasks but reaches only $60/68 = 88.2\%$ on the training tasks, so iterative reflection is useful but not monotonic.

This paper makes three contributions.

1. We formulate a capability-targeted adversarial co-evolution protocol for web-agent reflection rules: profile current failures, generate drifted websites at the boundary, audit task validity, mine behavior gaps, and promote only compact rule updates that survive validation.

2. We materialize the protocol in WEBCOEVO, a standalone LINKDING runner with explicit task normalization, reset logic, prompt construction, rulebook selection, and legacy eval/trace JSONL export.

3. We report the upgraded evidence across LINKDING V1.45.0, website V2, and website V3: EXPEL-style rules help on the original site, but compact reflection rules are what preserve performance under generated website drift, with R1 strongest on the training tasks under website V3 and R1/R4 tied on the held-out validation tasks under website V3.

The claim boundary is deliberately narrow. We do not claim universal web-agent robustness, nor that later reflection-rule versions always improve. The evidence is single-site, and several drift families remain difficult on the held-out validation tasks under website V3. The contribution is an auditable loop that turns those failures into the next environment and rule-design targets.

2 Related work

From static web tasks to version-aware evaluation. Web-agent evaluation is moving from frozen, single-version tasks toward settings that better approximate deployment. Early benchmarks made browser control measurable with reproducible websites, DOM observations, screenshots, and action traces (Zhou et al., 2023; Deng et al., 2023; Koh et al., 2024). Newer benchmarks add live content, deterministic replicas of real websites, long-horizon chores, and user-centered assistance where instructions can be ambiguous or evolve over time (Pan et al., 2024; Garg et al., 2025; Miyai et al., 2025; Ye et al., 2025). This shift reflects a broader concern that apparent progress can be inflated by contamination, stale benchmarks, and weak agreement with human judgments (Xue et al., 2025a). Time-aware evaluation makes the issue especially clear: agents tuned on one web design may fail across historical interface versions (Ishmam and Marino, 2026). WEBCOEVO follows this version-aware direction, but treats website versions as a design variable rather than only a fixed test distribution.

Website evolution as task-semantic drift. Website change is not merely cosmetic. A valid task can keep the same user goal while changing the entry point, readiness timing, labels, workflow checkpoints, route equivalence, or login return behavior. Web-evolution and testing work already separate superficial page differences from semantic structure, dynamic UI state, visual presentation, workflow order, and authentication hazards (Shao et al., 2021; Mesbah et al., 2012; Macchi et al., 2021; Sanchez et al., 2011; Jonker et al., 2018). Recent interface-for-agent work takes the complementary route of exposing semantic component interfaces or structured GUI knowledge so agents depend less on brittle visual shortcuts (Qiu et al., 2026; Yu et al., 2026). Our drift labels—access, surface, content, runtime, process, structural, and functional—are therefore an operational language for valid completion-path changes, not a universal taxonomy of web evolution.

Experience loops for self-evolving agents. LLM-agent research is also shifting from one-shot prompting toward reusable experience. Reflection and experiential-learning methods convert trajectories into natural-language memories or rules (Shinn et al., 2023; Zhao et al., 2023). Re-

cent self-evolving-agent work broadens this pattern to memory, tools, policies, architectures, user interaction, and multi-agent feedback (ang Gao et al., 2025). In practical systems, the same trend appears as interaction-derived data, backward construction of instructions, distilled strategic principles, and policy updates from experience rather than static supervision alone (Su et al., 2025; Wu et al., 2025; Liang et al., 2025; Xue et al., 2025b). WEBCOEVO uses this idea narrowly: the base web agent remains fixed, while paired control/drift traces update a compact cross-version reflection rulebook.

Adaptive environments and agent-environment co-evolution. A parallel line of work treats environments as active curricula rather than passive test sets. Unsupervised environment design, replay-guided adversarial design, and compositional environment generation all use the learner's current capability to decide which tasks should be generated or replayed next (Dennis et al., 2020; Jiang et al., 2021; Gur et al., 2021). Newer LLM-agent work extends this idea with synthetic interaction data and generative simulators that produce tasks near the agent's useful difficulty frontier (Su et al., 2025; Guo et al., 2025). WEBCOEVO brings this co-evolutionary view to web-agent robustness under stricter constraints: each generated LINKDING overlay must boot, preserve evaluator semantics, remain manually solvable, and expose a meaningful behavioral failure rather than an infrastructure artifact.

Positioning. These lines leave a missing middle layer. Version-aware benchmarks show that web-agent success is unstable across interface change; web-evolution work explains why such change is semantic; self-evolving-agent work shows how experience can become reusable guidance; and adaptive environment generation shows how the next task distribution can target an agent's frontier. WEBCOEVO connects them in one auditable loop: profile failures, generate valid website drift, mine paired trajectories, update compact reflection rules, and test whether the update preserves earlier successes.

3 Methodology

3.1 Adversarial Co-Evolution Loop

WEBCOEVO treats website generation and reflection-rule learning as a closed loop. At round k, a web agent with a fixed base prompt and an optional cross-version rulebook is evaluated on a control website and on generated website variants that preserve the user goal. The resulting trajectories are converted into a capability profile:

`can_do`, `borderline`, `cannot_do`, `control_unstable`, `invalid`.

We also compute paired transitions between a reference run and a candidate run: `both_success`, `saved`, `lost`, and `both_fail`. The `saved` rows show which rules help; `lost` rows are no-regression counterexamples; and `both_fail` rows define the next capability frontier.

The loop has two update paths. The environment path generates a harder website when a slice is saturated by the current rulebook. The rule path mines behavior gaps when the website is valid and failures are concentrated in recoverable agent behavior. A rule update is accepted only when it remains compact, avoids task-id-specific overfitting, preserves successful behaviors, and passes coverage checks on the benchmark task schema.

3.2 Standalone WebCoEvo Runner

The upgraded experiments use a standalone LINKDING runner rather than the older multi-harness research stack. The runner has four benchmark-facing stages. First, it loads training-task or held-out-validation JSON rows and normalizes each row into a stable schema with fields such as `task_id`, `source_task_id`, `drift_type`, `variant`, `family`, `version`, and `start_url`. Second, it launches the requested website version, resets seed data, handles localhost login and redirect recovery, and compiles each row into a BrowserGym task. Third, it builds the agent prompt from separated blocks: the task observation and goal, an optional EXPEL task-experience block, and an optional reflection-rule adaptation block. Fourth, it exports eval and trace JSONL so earlier analysis tools can read the new runs.

This separation prevents a common ambiguity in reflection-rule experiments. A rulebook file can exist on disk without actually being injected into the agent prompt. The WebCoEvo traces therefore record `injected_rule_ids` for the EXPEL layer and `cross_version_reflection_rule_ids` for the reflection-rule layer separately, along with the selected rulebook path, selection context, miss reasons, and reset-time errors. The website V3

headline runs are interpreted only through this trace-audited path.

3.3 Website Versions and Drift Taxonomy

All websites are derived from LINKDING v1.45.0. LINKDING V1.45.0 is the unmodified release. Website V2 is a moderately modified generated drift suite used to test whether R1 reflection rules transfer beyond the original site. Website V3 is the harder validation layer and contains seven drift labels:

$$\{\texttt{access}, \texttt{surface}, \texttt{content}, \texttt{runtime}, \texttt{process}, \texttt{structural}, \texttt{functional}\}.$$

The labels are operational. `access` shifts authentication or redirect preservation. `surface` changes visual framing and salience. `content` changes labels, placeholders, or nearby copy while preserving the data model. `runtime` changes readiness timing. `process` changes visible checkpoints in a workflow. `structural` regroups navigation affordances or route entrypoints. `functional` changes exposed feature affordances or route equivalence while keeping the task evaluable from visible state. This completion-path view is grounded in prior work on semantic web evolution, GUI properties, dynamic Ajax state changes, workflow testing, and heterogeneous login flows (Shao et al., 2021; Macchi et al., 2021; Mesbah et al., 2012; Sanchez et al., 2011; Jonker et al., 2018).

Each generated website must pass design gates before it is used for rule mining: it should break one primary agent assumption, keep the semantic user goal unchanged, remain manually solvable, and preserve evaluator validity. Failures caused by reset, parser, port, or evaluator problems are labeled as infrastructure issues rather than rule-learning evidence.

3.4 Rulebooks and Reflection Updates

EXPEL rules are task-experience rules induced from failed-then-success episodes. They often contain concrete operational details such as known credentials, target pages, and completion cues. Reflection rules are intentionally compact and scoped to drift patterns rather than individual row ids.

The main reflection-rule sets are successive rule-revision rounds obtained by iteratively reflecting on observed failures:

- **R1:** the strongest robust rule set in the upgraded results. It emphasizes login-next preservation, stale-bid recovery, direct route fallback, finalization when target evidence is already visible, collapsed-navigation recovery, and non-interactive element rejection.
- **R4:** a GPT-5.4-assisted update derived from R1 transition evidence. It edits two rules without adding or dropping rules, targeting login-next preservation and finalization when URL, query, heading, or redirected destination already proves completion.
- **R2:** a staged repair candidate that strengthens access and surface recovery but loses held-out robustness on website V3.
- **R3:** a selective merge candidate that improves over EXPEL-style rules on website V3 but does not dominate R1.

The R4 pipeline illustrates the WebCoEvo rule path. Training-task matched transitions from website V2 with R1 to website V3 with R1 contain 65 `both_success`, 2 `lost`, 0 `saved`, and 1 `both_fail` case. The candidate generation stage compresses these cases into constrained edit proposals, then deterministic verification checks the rulebook schema and task coverage. The resulting R4 candidate covers $68/68$ training-task rows, but its evaluation shows why promotion cannot rely on plausibility alone: it ties R1 on the held-out validation tasks under website V3 yet is weaker than R1 on the training tasks under website V3.

3.5 Evaluation Slices and Metrics

We evaluate two task banks. The training split has 68 rows and is near the rule-mining distribution. The held-out validation split has 167 rows and covers unseen task families. For a slice S, success rate is

$$\mathrm{SR}(S) = \frac{1}{|S|} \sum_{x \in S} \mathbf{1}[\mathrm{success}(x)].$$

We report per-drift rates, aggregate rates, and cross-version comparisons across website versions. The agent configuration is Qwen/Qwen3-VL-30B-A3B-Instruct with visual input enabled, `MAX_STEPS`=30, and `MAX_TOKENS`=300, using the `vl_action_reflection` mode through an OpenAI-compatible endpoint.

3.6 Claim Boundary

The upgraded method supports three claims. First, EXPEL task-experience rules are valuable on the

original site but lose robustness under harder website drift. Second, compact reflection rules recover much of that lost performance and transfer to the held-out validation tasks. Third, rule iteration is not monotonic: R4 is evidence-aware and trace-audited, but R1 remains the strongest rule set on the training tasks under website V3. The method does not yet establish robustness beyond LINKDING or prove that a single rulebook dominates every drift family.

4 LINKDING Experimental Setup

4.1 Task Banks

The training split is the calibration anchor for WebCoEvo. It contains 68 rows over task families chosen because they expose failures in login redirects, filtered views, prefilled forms, and navigation under website drift. We treat it as training-like because the reflection-rule loop is diagnosed and edited near this distribution. The held-out validation split contains 167 rows over unseen task families. We report the two task banks separately because averaging them would mix a rule-mining slice with a transfer slice.

4.2 Website Suites

The upgraded evaluation uses three LINKDING website suites.

- **LINKDING V1.45.0:** the original unmodified LINKDING release, used to measure the value of no rules versus EXPEL task-experience rules.
- **Website V2:** a moderately modified generated website suite, used to compare EXPEL-style prompting with R1 reflection rules.
- **Website V3:** the harder generated website suite, with seven release-grounded website overlays for `access`, `surface`, `content`, `runtime`, `process`, `structural`, and `functional` drift.

Website V2 is useful because it shows that R1 improves transfer before the harder generated website suite. Website V3 is useful because it exposes a sharper capability boundary: EXPEL-style prompting remains strong on earlier websites but drops sharply under this harder generated suite.

4.3 Agent and Prompt Conditions

All headline runs use Qwen/Qwen3-VL-30B-A3B-Instruct with visual input enabled, `vl_action_reflection` agent mode, `MAX_STEPS`=30, and `MAX_TOKENS`=300. The prompt can contain three separated sources of information: the current observation and goal, EXPEL task-experience rules, and reflection rules. The LINKDING V1.45.0 matrix compares no rules with EXPEL-style rules. The website V2 matrix compares EXPEL-style rules with R1 reflection rules. The website V3 matrix compares EXPEL-style rules with R1–R4 reflection rules.

4.4 Rulebook Selection and Auditing

The website V3 comparison evaluates each reflection-rule set as a compact whole policy with coarse drift matching. It does not use source-task-id gating in the headline comparison. The runner records prompt injection explicitly: `injected_rule_ids` for EXPEL rules and `cross_version_reflection_rule_ids` for reflection rules. It also records rulebook paths, selection context, miss reasons, and reset-time errors. This lets the results be interpreted as rule-conditioned runs rather than as silent no-rule reruns.

4.5 Provenance

The upgraded data come from completed WebCoEvo reports generated on April 17-18, 2026 under `/home/gecm/WebCoEvo`. The key report files are the website V3 reflection-rule matrix report, the UMich Qwen3-VL rule-comparison report, the website-version line report, and the GPT-5.4 R4 reflection-hardening report. Appendix Table 4 lists the local artifacts used to update the paper.

5 Results

Experimental setting. We evaluate a fixed multimodal web agent on a self-hosted bookmark-management website under three conditions: no retrieved rules, EXPEL-style rules (Zhao et al., 2023), and EXPEL-style rules augmented with reflection rules. The evaluation uses 68 training tasks for diagnosing rule behavior and 167 held-out validation tasks for testing transfer to unseen task families. We report task success rate on LINKDING V1.45.0, on a moderately modified website V2, and on a harder generated website suite V3 that changes access, visual presentation, content, timing, work-

Training tasks

Website	No rules	EXPEL-style	Reflection
V1.45.0	9/68	56/68	–
V2	–	60/68	67/68
V3	–	8/68	65/68

Held-out validation tasks

Website	No rules	EXPEL-style	Reflection
V1.45.0	133/167	124/167	–
V2	–	114/167	143/167
V3	–	66/167	97/167

Table 1: Overall success counts across website versions. EXPEL-style rules help on LINKDING V1.45.0 but degrade under harder website drift. Reflection rules remain stronger on the generated website suites. Detailed per-drift results for R1–R4 on website V3 are reported in Appendix Tables 5 and 6.

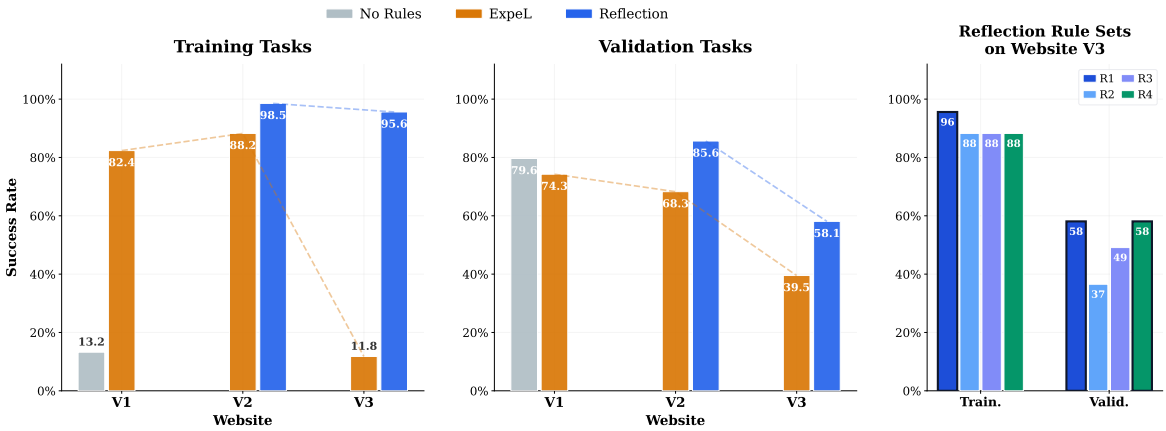

Figure 1: Headline robustness results across website versions. The left and middle panels compare no rules, EXPEL-style rules, and the strongest available reflection-rule condition on the training and held-out validation tasks. The right panel isolates the website V3 reflection-rule sets, showing that later rule revisions are not monotonically better.

Rules	Training	Held-out
R1	65/68 (95.6%)	97/167 (58.1%)
R2	60/68 (88.2%)	61/167 (36.5%)
R3	60/68 (88.2%)	82/167 (49.1%)
R4	60/68 (88.2%)	97/167 (58.1%)

Table 2: Overall website V3 success for successive reflection-rule sets. R1–R4 are obtained through iterative reflection on observed failures and rule revision; detailed per-drift counts are deferred to Appendix Tables 5 and 6.

flow, navigation structure, and feature affordances while preserving the user goal. For website V3, R1–R4 denote successive reflection-rule sets obtained through iterative reflection on observed failures and rule revision.

5.1 EXPEL-Style Rules Help, but Do Not Fully Transfer

On LINKDING V1.45.0, EXPEL-style rules provide a strong training-task gain: success increases from 13.2% without rules to 82.4% with retrieved EXPEL-style rules. This confirms that reusable experience is a meaningful starting point rather than a weak baseline. The same rules, however, do not uniformly improve held-out behavior. On held-out validation tasks from LINKDING V1.45.0, success changes from 79.6% without rules to 74.3% with EXPEL-style rules. This motivates separating training tasks from held-out validation tasks instead of reporting a single averaged score.

5.2 Reflection Rules Help Under Moderate Website Drift

We next test whether reflection rules add value when the website changes but the task goals remain the same. On website V2, adding reflection rules improves the training tasks from 88.2% to 98.5%, and improves the held-out validation tasks from 68.3% to 85.6% (Table 1 and Figure 1). Since the model, tasks, website version, and EXPEL-style rules are held fixed, these gains isolate the benefit of rules that describe how to preserve intent across interface changes.

5.3 Website V3 Exposes the Robustness Gap

The harder generated website suite V3 creates the main robustness stress test. With EXPEL-style rules alone, success drops to 11.8% on the training

tasks. The strongest reflection-rule set recovers performance to 95.6%, while three alternative rule sets reach 88.2% (Table 2). Figure 10 shows that the recovery is broad across drift families rather than coming from a single easy bucket. This large gap shows that robustness under website drift is not obtained simply by adding more task-level guidance; the useful rules are those that explicitly describe how to recover from changed navigation paths, delayed readiness, changed labels, and shifted workflow cues.

The held-out validation tasks remain substantially harder. EXPEL-style rules alone reach 39.5%. The two strongest reflection-rule sets, R1 and R4, reach 58.1%, while weaker variants R3 and R2 reach 49.1% and 36.5%. Thus, reflection rules improve transfer under harder drift, but the gains are uneven: a plausible rule update can improve one family of failures while regressing another.

5.4 Rule Updates Are Useful but Non-Monotonic

The detailed website V3 matrices in Appendix Tables 5 and 6, summarized visually in Figure 8 and Figure 10, show that the best aggregate reflection-rule set is not simply the most recent edit. One evidence-based update improves several held-out drift categories, including authentication-related, visual, content, and timing changes, but loses ground on workflow and feature-affordance changes. The aggregate tie on the held-out validation tasks therefore hides a real tradeoff between different recovery behaviors.

This pattern supports the main design choice in WEBCOEVO: rule generation, verification, and promotion should be separate stages. Matched trajectories can suggest useful edits, but an edit should not be promoted only because it is plausible or locally evidence-grounded. It must also preserve previously successful behaviors under the same website-drift distribution.

5.5 Failure Analysis: Why Iterative Reflection Degrades Held-Out Generalization

The per-drift breakdown in Appendix Tables 5 and 6 exposes a sharp asymmetry between in-distribution repair and held-out transfer. On the 68 training tasks, successive rule sets differ modestly: R1 reaches $65/68$, while R2–R4 each reach $60/68$. On the 167 held-out tasks, the same iteration path is much less stable: R1 reaches $97/167 = 58.1\%$, R2 falls to $61/167 = 36.5\%$, and R3–R4 recover to $82/167 = 49.1\%$ and $97/167 = 58.1\%$. The R1→R2 transition loses only five training tasks but thirty-six held-out tasks, an approximately $7\times$ larger loss on the transfer split. This gap suggests that the main failure is not weak optimization on the observed traces, but a mismatch between the local evidence used to edit rules and the broader conditions under which those rules are later applied.

Reflection edits trade specificity for coverage. Across R1–R4, the strongest rules are not the most detailed ones; they are the ones that preserve abstract completion evidence across many website states. R1 succeeds because several rules describe invariants such as destination preservation, query-state evidence, and visible target confirmation. R2, by contrast, often replaces broad stopping conditions with narrower procedural advice targeted at the most recent training failures. These edits are locally rational: they explain why the agent failed on the observed trajectory. They are also globally risky: when a rule changes from "what state proves completion" to "what action to take at this checkpoint," it no longer protects held-out tasks whose surface workflow differs but whose completion evidence is already sufficient. The resulting degradation is concentrated in `process` and `functional` tasks, where agents must decide when a multi-step interaction has already satisfied the goal rather than merely find the next clickable element.

Observable failures recover faster than semantic failures. Later iterations partially repair categories whose errors leave direct page-level evidence. Access and surface drift provide visible signals such as a wrong redirect, missing entry point, or changed navigation affordance; these signals are easy for a trajectory-to-rule editor to identify and rephrase. This helps explain why R3 and R4 improve the held-out `access`, `surface`, and `runtime` buckets relative to R2. The same process is weaker for `process`, `functional`, and `structural` drift. In these cases, success depends on an abstract relation between goal state, route state, and task semantics. Once an iteration-specific patch displaces that abstraction, subsequent trace evidence may show repeated failure but not reveal the missing principle unless the prompt explicitly asks the generator to preserve previously successful stopping behavior.

Content drift reveals a different limitation. The `content` bucket is the only held-out drift family where EXPEL-style rules remain stronger than every reflection-augmented rulebook: EXPEL reaches $11/14 = 78.6\%$, while R1–R4 range from $5/14$ to $10/14$. This is not the same failure as the R1→R2 process collapse. Content drift changes labels, placeholders, and nearby wording while keeping the data model intact, so robustness requires tolerance to lexical variation. The EXPEL-style layer, distilled from broader task experience, appears to encode this tolerance as a diffuse behavioral prior. The reflection layer is more brittle because its evidence comes from a narrower set of generated content variants; even when a rule is written in general language, its implicit anchors can remain tied to the strings and form contexts seen during mining.

Implication for reflection-rule learning. The results point to a generalization problem in the rule editor rather than a failure of reflection as a mechanism. Reflection improves V3 robustness substantially, but unconstrained iteration can convert transferable principles into high-precision patches. A safer promotion criterion should therefore evaluate three properties separately: whether an edit explains the new failure, whether it preserves the old success conditions that the replaced rule covered, and whether its predicates are expressed in observable state evidence rather than task-specific surface tokens. In other words, the reflection loop should treat rule updates as policy changes with regression risk, not as monotonic memory accumulation. Appendix A provides the trace-level case studies and rule diffs that ground these mechanisms.

6 Discussion and Conclusion

The upgraded WebCoEvo evidence changes the paper's center of gravity. The earlier version treated the final benchmark as a narrower website-drift ablation with unresolved held-out access evidence. The current result is cleaner: the same standalone runner reports complete 68-row training and 167-row held-out validation comparisons across LINKDING V1.45.0, website V2, and website V3. That makes the empirical claim both stronger and more constrained.

The strongest methodological point is trace-audited separation of rule layers. EXPEL rules and reflection rules are not treated as one opaque prompt. The runner records which task-experience rules were injected, which reflection rules were injected, which rulebook path was selected, and whether failures occurred during reset or during the agent rollout. This evidence standard matters because a reflection-rule paper can otherwise confuse a rulebook existing on disk with a rule actually reaching the model.

The strongest empirical point is that website drift changes which kind of rule is useful. On the training tasks from LINKDING V1.45.0, EXPEL rules are extremely helpful. On the training tasks under website V3, the same class of task-experience rules collapses to $8/68 = 11.8\%$, while R1 reflection rules reach $65/68 = 95.6\%$. On the held-out validation tasks under website V3, R1 and R4 tie at $97/167 = 58.1\%$, improving over EXPEL-style rules at $66/167 = 39.5\%$. The lesson is not that task-experience rules are bad. It is that task-experience rules and reflection rules solve different parts of the adaptation problem.

Rule refinement. The natural next rulebook should start from R1, not simply from the latest merge. R1 remains the strongest rule set on the training tasks under website V3 and ties the best held-out validation score. R4 contributes useful evidence about login-next preservation and finalization, but its training-task regression means those edits need a narrower promotion gate. A good next version should preserve R1's process and functional behavior while selectively borrowing R4's access, surface, content, and runtime gains.

Reflection prompt optimization (future work). The failure analysis in Section 5.5 identifies the reflection generator's prompt design as the primary lever for improving iterative rule quality. Several directions are worth exploring. First, *generality-constrained generation*: the prompt should explicitly instruct the generator to express rules in terms of observable page-state evidence (DOM structure, URL patterns, heading text) rather than task-specific route strings or label tokens that may not appear in held-out tasks. Second, *no-regression rails*: each iteration's prompt should include a curated set of positive trajectory examples from held-out proxy tasks alongside training failures, with an explicit instruction to verify that proposed edits do not weaken rule behaviors that were already working. Third, *category-isolated promotion*: running separate reflection passes per drift family and applying a cross-family interference check before any edit is merged would contain the

`process/functional` degradation observed in R2. Fourth, *held-out gating in the loop*: sampling a small proxy held-out set at each iteration and rejecting candidate rule sets that regress on it would prevent the catastrophic R1→R2 drop without requiring a full held-out evaluation at each step. Together, these changes shift the reflection loop from a locally evidence-grounded editor into a generalization-aware rule curator.

Environment hardening. The environment generator should remain capability-gated. Saturated drift types should be hardened by breaking one primary assumption while preserving task semantics and evaluator validity. Frontier drift types should be held steady for rule validation. Infrastructure failures should trigger runner or evaluator repair, not rule mining. This keeps the co-evolution loop honest: the next website should expose an agent limitation, not manufacture an invalid task.

Cross-site transfer. The open claim is whether WebCoEvo is a LINKDING repair procedure or a reusable web-agent adaptation protocol. The next transfer step should use a second local web application with deterministic reset, credentials, seed data, and stable evaluators. We would map the same seven drift categories into that site, run EXPEL-style versus compact reflection-rule baselines, and then test whether the mined rules remain general or become site-specific.

Limitations. The current evidence remains single-site. The website versions are generated from one application family, and the strongest held-out validation score under website V3 is still only 58.1%. Some rule edits improve one drift family while regressing another, and aggregate scores can hide those tradeoffs. Finally, the method depends on trace quality: if the runner fails to expose reset errors, rule-selection misses, or prompt-injection state, the co-evolution loop can mine the wrong evidence.

In conclusion, WEBCOEVO turns web-agent website drift from a static benchmark into an auditable improvement loop. It profiles failures, generates capability-targeted LINKDING website versions, separates task-experience and reflection rules, and shows that compact reflection rules recover substantial performance under website V3 drift. The current best evidence is R1: $65/68 = 95.6\%$ on the training tasks under website V3 and $97/167 = 58.1\%$ on the held-out validation tasks under website V3. The remaining work is concrete: preserve R1's robust behaviors, repair the drift-family regressions exposed by R4, and test the same protocol beyond LINKDING.

References

Huan ang Gao, Jiayi Geng, Wenyue Hua, Mengkang Hu, Xinzhe Juan, Hongzhang Liu, Shilong Liu, Jiahao Qiu, Xuan Qi, Yiran Wu, Hongru Wang, Han Xiao, Yuhang Zhou, Shaokun Zhang, Jiayi Zhang, Jinyu Xiang, Yixiong Fang, Qiwen Zhao, Dongrui Liu, and 8 others. 2025. A survey of self-evolving agents: On path to artificial super intelligence. *arXiv preprint*.

Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. 2023. Mind2web: Towards a generalist agent for the web. *arXiv preprint*.

Michael Dennis, Natasha Jaques, Eugene Vinitsky, Alexandre M. Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. 2020. Emergent complexity and zero-shot transfer via unsupervised environment design. In *Advances in Neural Information Processing Systems 33*.

Divyansh Garg, Shaun VanWeelden, Diego Caples, Andis Draguns, Nikil Ravi, Pranav Putta, Naman Garg, Tomas Abraham, Michael Lara, Federico Lopez, James Liu, Atharva Gundawar, Prannay Hebbar, Youngchul Joo, Jindong Gu, Charles London, Christian Schroeder de Witt, and Sumeet Motwani. 2025. Real: Benchmarking autonomous agents on deterministic simulations of real websites. *arXiv preprint*.

Jiacheng Guo, Ling Yang, Peter Chen, Qixin Xiao, Yinjie Wang, Xinzhe Juan, Jiahao Qiu, Ke Shen, and Mengdi Wang. 2025. Genenv: Difficulty-aligned co-evolution between llm agents and environment simulators. *arXiv preprint*.

Izzeddin Gur, Natasha Jaques, Yingjie Miao, Jongwook Choi, Manoj Tiwari, Honglak Lee, and Aleksandra Faust. 2021. Environment generation for zero-shot compositional reinforcement learning. In *Advances in Neural Information Processing Systems 34*, pages 4157–4169.

Md Farhan Ishmam and Kenneth Marino. 2026. Timewarp: Evaluating web agents by revisiting the past. *arXiv preprint*.

Minqi Jiang, Michael Dennis, Jack Parker-Holder, Jakob N. Foerster, Edward Grefenstette, and Tim Rocktäschel. 2021. Replay-guided adversarial environment design. In *Advances in Neural Information Processing Systems 34*, pages 1884–1897.

Hugo Jonker, Jelmer Kalkman, Benjamin Krumnow, Marc Sleegers, and Alan Verresen. 2018. Shepherd: Enabling automatic and large-scale login security studies. *arXiv preprint*.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. 2024. Visualwebarena: Evaluating multimodal agents on realistic visual web tasks. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905. Association for Computational Linguistics.

Xuechen Liang, Meiling Tao, Yinghui Xia, Jianhui Wang, Kun Li, Yijin Wang, Yangfan He, Jingsong Yang, Tianyu Shi, Yuantao Wang, Miao Zhang, and Xueqian Wang. 2025. Sage: Self-evolving agents with reflective and memory-augmented abilities. *Neurocomputing*, 647:130470.

Federico Macchi, Pierpaolo Rosin, Juan Marcos Mervi, and Luca Turchet. 2021. Image-based approaches for automating gui testing of interactive web-based applications. In *2021 28th Conference of Open Innovations Association (FRUCT)*, pages 278–285. IEEE.

Ali Mesbah, Arie van Deursen, and Stefan Lenselink. 2012. Crawling ajax-based web applications through dynamic analysis of user interface state changes. *ACM Transactions on the Web*, 6(1):1–30.

Atsuyuki Miyai, Zaiying Zhao, Kazuki Egashira, Atsuki Sato, Tatsumi Sunada, Shota Onohara, Hiromasa Yamanishi, Mashiro Toyooka, Kunato Nishina, Ryoma Maeda, Kiyoharu Aizawa, and Toshihiko Yamasaki. 2025. Webchorearena: Evaluating web browsing agents on realistic tedious web tasks. *arXiv preprint*.

Yichen Pan, Dehan Kong, Sida Zhou, Cheng Cui, Yifei Leng, Bing Jiang, Hangyu Liu, Yanyi Shang, Shuyan Zhou, Tongshuang Wu, and Zhengyang Wu. 2024. Webcanvas: Benchmarking web agents in online environments. *arXiv preprint*.

Zhi Qiu, Jiazheng Sun, Chenxiao Xia, Jun Zheng, and Xin Peng. 2026. Ci4a: Semantic component interfaces for agents empowering web automation. *arXiv preprint*.

Mario E. Sanchez, Camilo H. Jimenez, and Jorge A. Villalobos. 2011. Model based testing for workflow enabled applications. *Computacion y Sistemas*, 14(4):335–350.

Fei Shao, Rui Xu, Wasif Arman Haque, Jingwei Xu, Ying Zhang, Wei Yang, Yanfang Ye, and Xusheng Xiao. 2021. Webevo: taming web application evolution via detecting semantic structure changes. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 16–28. ACM.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint*.

Hongjin Su, Ruoxi Sun, Jinsung Yoon, Pengcheng Yin, Tao Yu, and Sercan Ö. Arık. 2025. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. In *International Conference on Learning Representations*.

Rong Wu, Xiaoman Wang, Jianbiao Mei, Pinlong Cai, Daocheng Fu, Cheng Yang, Licheng Wen, Xuemeng Yang, Yufan Shen, Yuxin Wang, and Botian Shi. 2025. Evolver: Self-evolving llm agents through an experience-driven lifecycle. *arXiv preprint*.

Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025a. An illusion of progress? assessing the current state of web agents. *arXiv preprint*.

Xiangyuan Xue, Yifan Zhou, Guibin Zhang, Zaibin Zhang, Yijiang Li, Chen Zhang, Zhenfei Yin, Philip Torr, Wanli Ouyang, and Lei Bai. 2025b. Comas: Co-evolving multi-agent systems via interaction rewards. *arXiv preprint*.

Suyu Ye, Haojun Shi, Darren Shih, Hyokun Yun, Tanya Roosta, and Tianmin Shu. 2025. Realwebassist: A benchmark for long-horizon web assistance with real-world users. *arXiv preprint*.

Mingxian Yu, Siqi Luo, and Xu Chen. 2026. Graphpilot: Gui task automation with one-step llm reasoning powered by knowledge graph. *arXiv preprint*.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2023. Expel: Llm agents are experiential learners. *arXiv preprint*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2023. Webarena: A realistic web environment for building autonomous agents. *arXiv preprint*.

A Trace-Level Case Studies for Reflection-Rule Degradation

The main text analyzes reflection-rule degradation at the level of rule-learning mechanisms. This appendix keeps the concrete audit trail: rule substitutions, task families, trajectory behavior, and error modes extracted from the evaluation JSONL logs.

Table 3: Case-study map for the failure mechanisms in Section 5.5.

Case	Local edit	Observed failure	Main-text mechanism
A	Replaces a broad finalization rule with staged-checkpoint advice.	Held-out process tasks repeat form-submission cycles until `max_steps_no_success`.	A locally valid procedural patch removes a transferable completion invariant.
B	Tightens query-state completion into a prepared-versus-committed distinction.	Held-out functional tasks continue acting after the target filter state is reached.	Extra specificity creates hesitation when unseen variants do not expose the same intermediate state.
C	Reflection rules are mined from narrow content variants.	Held-out content tasks fail despite correct navigation, while EXPEL-style rules remain stronger.	Narrow evidence can encode implicit surface-token anchors even when rules are phrased generally.

A.1 Case A: Finalization Rule Removal Causes Infinite Loop on Process Tasks

Task and setting. Task `TKB04_EDIT_TITLE_AMBER` (task ID 1502505) is a `process`-drift held-out task that asks the agent to create a bookmark with a specific edited title on website V3. The V3 process drift introduces a staged checkpoint: the bookmark creation form is served under the URL `/bookmarks/new?hardv3_release_ready=1`, signaling that the form has passed an internal readiness gate before the agent may submit. Successful completion requires the agent to recognize that the post-submission landing at `/bookmarks` constitutes final state and report the created bookmark's title.

R1 trajectory (success, 13 steps). Under R1, the agent navigates to the form, fills the URL, tag, and title fields, and submits (steps 0–5). After the submission lands at `/bookmarks`, the agent makes one additional pass to refine the title field to its final form (steps 6–10), then issues `send_msg_to_user("TaskBank Amber 1450 Edited tb1450-amber")` at step 11. The agent's decision to finalize is consistent with the injected R1 rule: *"Finalize instead of noop when URL, query, or heading already proves the target state."* The `/bookmarks` landing page with the bookmark visible serves as that proof.

R2 trajectory (failure, 31 steps, `max_steps_no_success`). Under R2, the agent follows an identical initial sequence (steps 0–5), successfully submitting the form and landing at `/bookmarks`. However, instead of issuing a final answer, it immediately returns to `/bookmarks/new` and begins a second fill-click-submit cycle (steps 6–9). This pattern—navigate to form, fill URL, click submit, land at `/bookmarks`, navigate back—repeats verbatim for seven additional cycles before the step budget is exhausted at step 30. The agent never issues `send_msg_to_user`.

Root cause. Inspecting the R2 rulebook reveals that the rule responsible for completion detection in R1 (`xvr24_0003`, covering seven drift types) was replaced by *"Treat staged review or apply pages as one-click checkpoints, not as noop states"* (`xvr25_0003`). This substitution addresses a genuine training failure—agents looping on staged review pages—but it describes *how* to handle a checkpoint, not *when* the overall task is complete. Without a general finalization principle, the agent cannot distinguish the successful `/bookmarks` landing from an intermediate state, and the "hard stop on repeated noop or goto" rule in R2 (`xvr25_0008`) does not trigger because each cycle involves distinct fill and click actions rather than identical noops. The same failure pattern appears across all thirteen process-drift held-out task instances that R1 previously handled, accounting for the full $8/19$ gap in held-out process success between R1 ($13/19$) and R2 ($5/19$).

A.2 Case B: Completion-Signal Tightening Causes Hesitation on Functional Tasks

Task family and rule comparison. Functional-drift tasks on the held-out set include several families requiring the agent to navigate to a filtered bookmark view and confirm the filter state. R1 provides *"For filtered bookmark tasks, treat query-state arrival as completion evidence"* (`xvr24_0007`, covering `content`, `runtime`, `functional`, and `surface` drift). Under this rule, reaching the URL `/bookmarks?q=`*<tag>* is sufficient for the agent to issue a final answer.

The R2 iteration observed training cases in which the agent prematurely finalized on a *prepared* filter state—a view where the query string was pre-populated but the filter results were not yet committed—and proposed a corrective rule: *"For filtered bookmark tasks, distinguish prepared lookup states from committed filter results"* (`xvr25_0007`). This rule is locally sound for the training failure it targets, but it introduces a two-state distinction that held-out task variants do not expose: in the held-out functional tasks, the filtered URL already represents the committed result, so the additional evidence the agent is now instructed to seek does not appear. The agent therefore continues interacting after the target state is already reached, exhausting its step budget. Held-out functional success drops from $7/13 = 53.8\%$ under R1 to $3/13 = 23.1\%$ under R2. R4 explicitly restores the R1 completion phrasing, recovering to $4/13 = 30.8\%$, though not to R1 levels, as other rule differences retain some residual hesitation.

A.3 Case C: Content-Token Specificity Explains Persistent ExpeL Advantage

Observation. The `content`-drift bucket is the only drift family where EXPEL-style rules ($11/14 = 78.6\%$) outperform every reflection-augmented version across all four iterations: R1 ($7/14 = 50.0\%$), R2 ($5/14 = 35.7\%$), R3 ($6/14 = 42.9\%$), and R4 ($10/14 = 71.4\%$).

Failure pattern. Content drift changes labels, placeholders, and surrounding wording while preserving the underlying data model and task structure. The failing held-out task families (primarily `TKB05_EDIT_TAGS_QUERY` and `TKB10_NOSCHEME_POLICY` variants) exhibit content changes that do not appear in any training-task trajectory. Inspection of the eval records for R1 content failures shows that all seven failing tasks share the same error (`max_steps_no_success`) despite the agent receiving seven reflection rules, including the finalization and stuck-policy rules that resolve other drift families. The agent correctly navigates to the relevant form but cannot reconcile the modified label or placeholder text with its expectations, resulting in repeated interaction cycles that never reach the evaluator's acceptance criterion.

Why ExpeL generalizes better. EXPEL-style rules were distilled from trajectories spanning many task families and website states, giving them implicit tolerance for surface-token variation. Reflection rules, generated from a small set of failure traces on specific training-content variants, encode expectations about the exact label strings or placeholder wording present in those traces. For held-out content variants that use different surface tokens while preserving task structure, these implicit expectations act as interference: the agent's behavior is constrained by assumptions that do not hold. This case illustrates a general limit of evidence-grounded reflection: the quality of the abstraction is bounded by the diversity of the evidence, and narrow evidence produces narrow rules even when the rule's stated scope appears broad.

B Artifact Notes

B.1 Source Reports

Table 4: Local WebCoEvo artifacts used to upgrade the paper.

Artifact	Role
Website V3 matrix	The April 17 website V3 reflection-rule matrix report supplies the training-task and held-out validation comparison for R1–R4 and EXPEL-style baselines.
Rule comparison	The April 18 UMich Qwen3-VL rule-comparison report supplies LINKDING V1.45.0 and website V2 comparisons for no rules, EXPEL-style rules, and R1 reflection rules.
Website trend	The April 18 website-version line report supplies the cross-website trend over LINKDING V1.45.0, website V2, and website V3.
R4 hardening	The April 18 GPT-5.4 hardening report supplies R1-to-R4 transition counts, edited rule summaries, and verification status.
Reflection rulebooks	The R1–R4 JSON rulebooks are the compact reflection policies evaluated in the website V3 matrix.
EXPEL rulebook	The official V2 EXPEL JSON rulebook is the fixed task-experience layer used for EXPEL-conditioned runs.
Run outputs	The WebCoEvo `results/` tree contains the eval and trace JSONL files consumed by the reports.

B.2 Full Website V3 Per-Drift Tables

Table 5: Training-task website V3 full benchmark.

Bucket	EXPEL-style	R1	R2	R3	R4
`access`	0/13	13/13	13/13	13/13	13/13
`surface`	1/13	13/13	13/13	13/13	12/13
`content`	2/9	7/9	5/9	4/9	6/9
`runtime`	2/16	16/16	15/16	16/16	16/16
`process`	0/6	6/6	5/6	5/6	5/6
`structural`	3/6	6/6	6/6	6/6	5/6
`functional`	0/5	4/5	3/5	3/5	3/5
Overall	**8/68 = 11.8%**	**65/68 = 95.6%**	**60/68 = 88.2%**	**60/68 = 88.2%**	**60/68 = 88.2%**

Table 6: Held-out validation website V3 full benchmark.

Bucket	EXPEL-style	R1	R2	R3	R4
`access`	6/36	8/36	3/36	12/36	10/36
`surface`	21/36	28/36	19/36	25/36	30/36
`content`	11/14	7/14	5/14	6/14	10/14
`runtime`	20/36	28/36	22/36	24/36	29/36
`process`	3/19	13/19	5/19	8/19	9/19
`structural`	2/13	6/13	4/13	4/13	5/13
`functional`	3/13	7/13	3/13	3/13	4/13
Overall	**66/167 = 39.5%**	**97/167 = 58.1%**	**61/167 = 36.5%**	**82/167 = 49.1%**	**97/167 = 58.1%**

B.3 Drift Taxonomy

Table 7: Operational definitions of the seven website shifts.

Shift	Completion-path node	Definition in this paper	Held constant
`access`	Authentication	Changes login, re-entry, or preservation of a requested `next` destination.	Credentials, destination semantics, evaluator target.
`surface`	Visible entrypoint	Changes visual framing, emphasis, or salience while keeping routes and controls stable.	Routes, control semantics, task end state.
`content`	Semantic content	Changes labels, placeholders, or nearby wording while preserving the data model.	Data model, URLs, task-specific entity strings.
`runtime`	Readiness and stop	Changes hydration, debounce, or rendering delay so the agent must re-check readiness.	Final DOM semantics, routes, goal state.
`process`	Workflow sequence	Changes visible review/checkpoint wording or ordering pressure in multi-step workflows.	Underlying state transition and postcondition.
`structural`	Navigation structure	Changes menu grouping, drawer organization, or entrypoint location.	Destination semantics and human reachability.
`functional`	Feature-route semantics	Changes exposed feature affordances or route equivalence while keeping the task evaluable.	Core site purpose, visible evaluability, task contract.

B.4 Website Modification Examples

The first modified website suite was produced as a vibe-coded LINKDING overlay before the harder generated website V3 suite. Figure 2 and Figure 3 show before/after examples for the seven drift families. The images are copied into the paper artifact so that this appendix is self-contained.

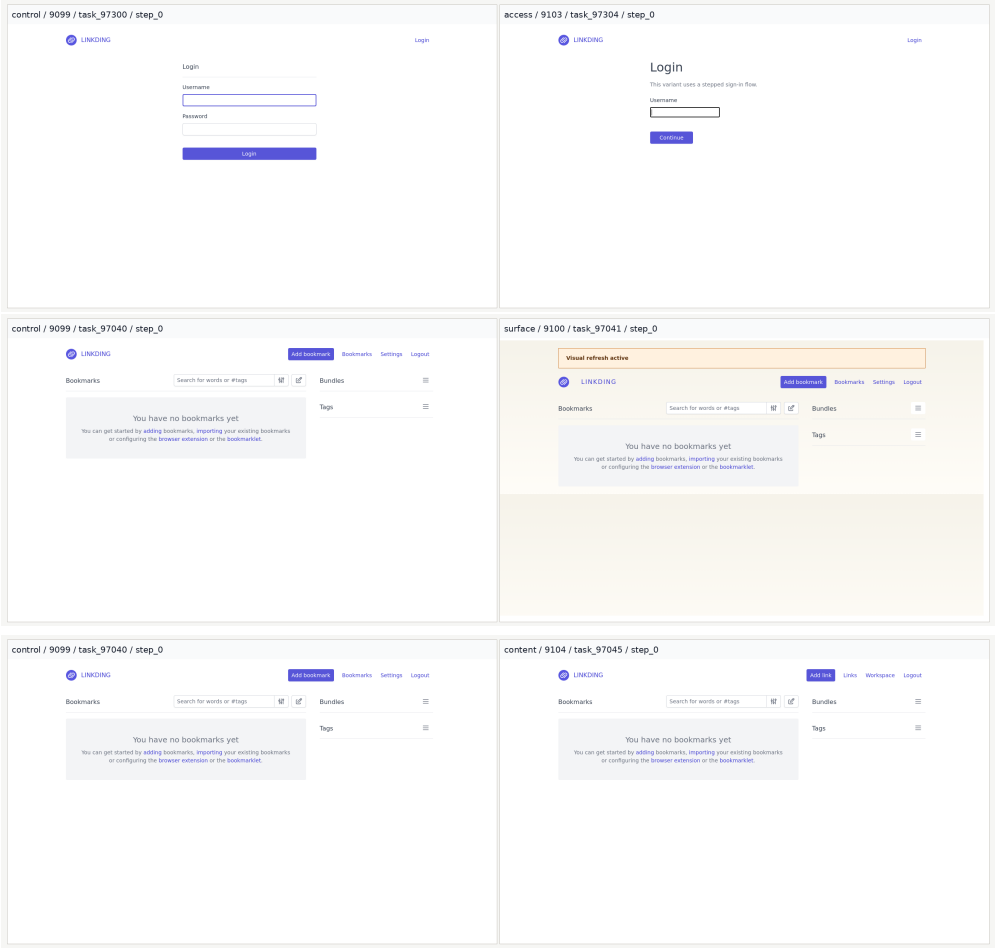

Figure 2: Examples from the first modified vibe-coded website suite: access, surface, and content drift. Each panel compares the original LINKDING interface with the modified website while preserving the benchmark goal.

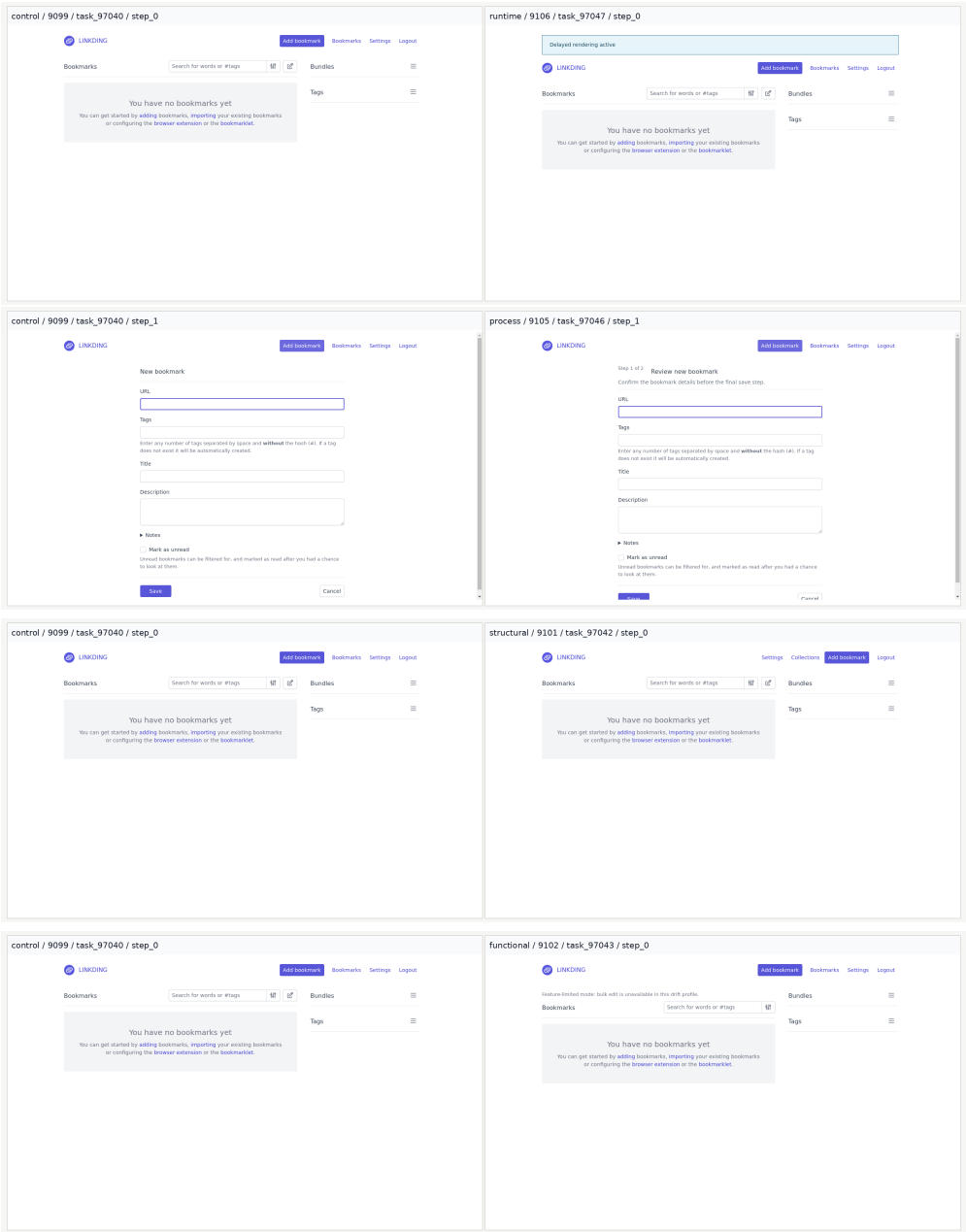

Figure 3: Examples from the first modified vibe-coded website suite: runtime, process, structural, and functional drift. These examples illustrate the kinds of interface changes used before the harder website V3 evaluation layer.

B.5 Additional Result Figures

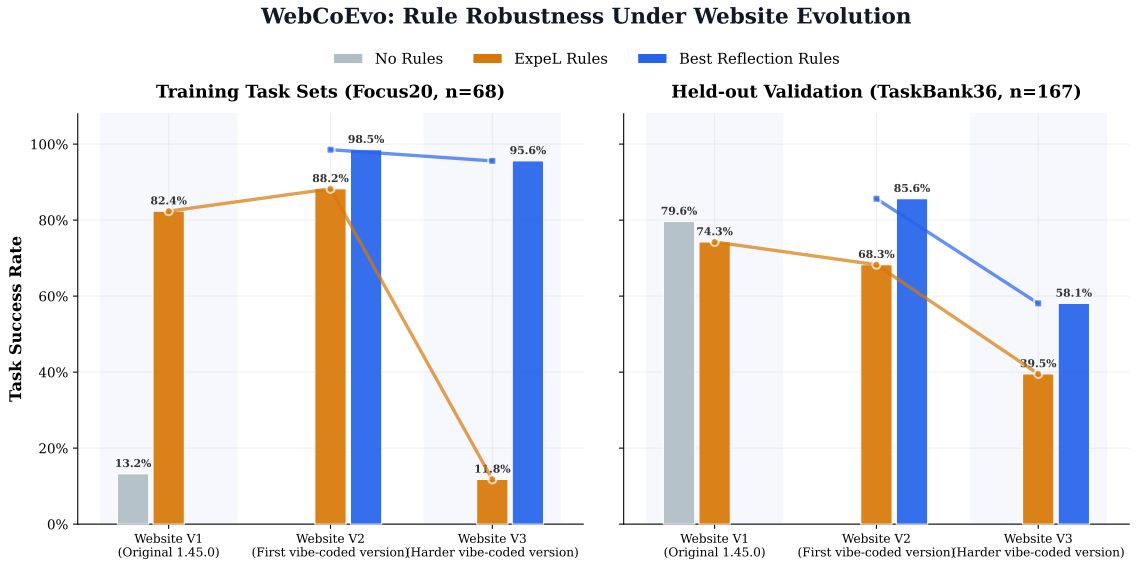

Figure 4: Condensed result panel used for paper and poster summaries. The panel places the main website-version comparison beside the website V3 reflection-rule comparison, making the robustness gap and non-monotonic rule-update pattern visible in one view.

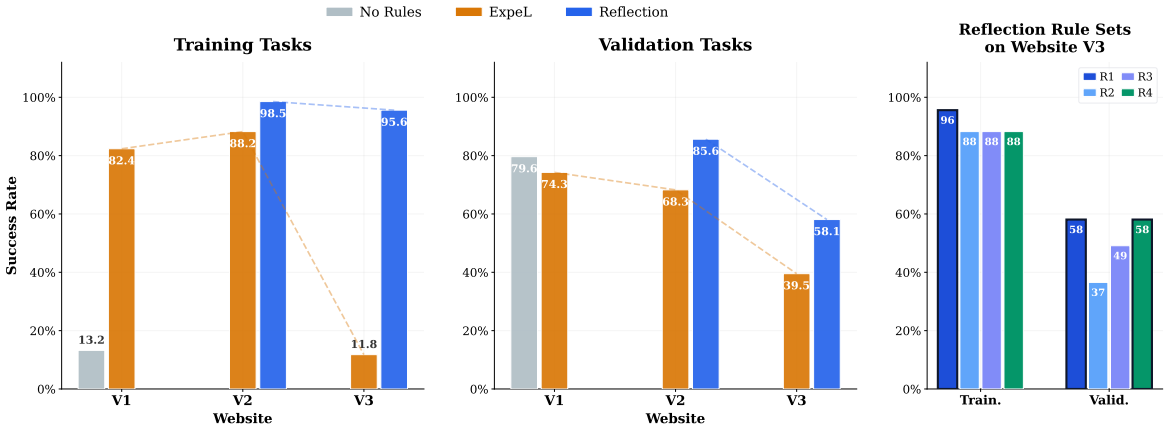

Figure 5: Poster headline figure for aggregate robustness results across website versions. This wider rendering mirrors Figure 1 and is included here as a complete artifact view.

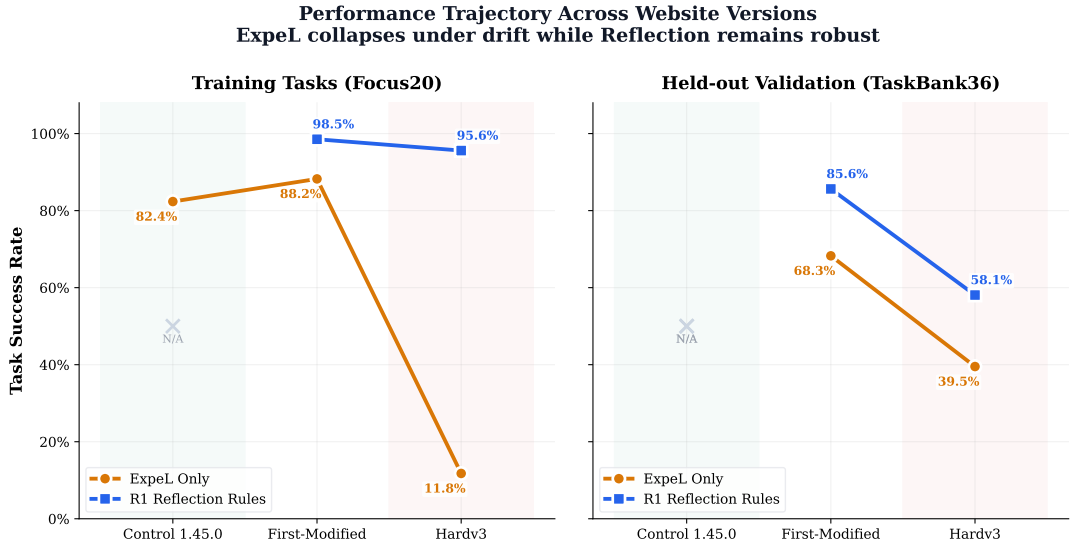

Figure 6: Version-level success trajectory across LINKDING V1.45.0, website V2, and website V3. The trajectory view emphasizes how EXPEL-style task experience and reflection rules respond differently as the website drift becomes harder.

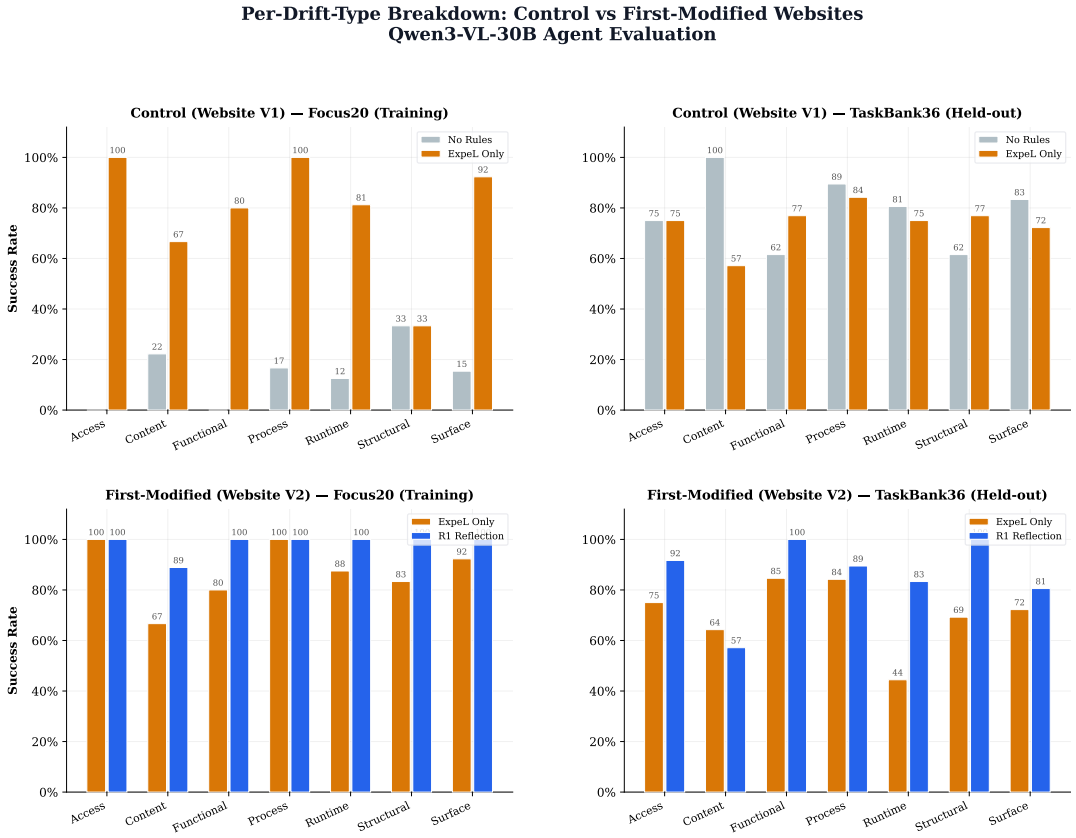

Figure 7: Control comparison between the unmodified LINKDING V1.45.0 condition and the first modified website condition. This figure isolates the initial effect of website drift before the harder generated website V3 suite.

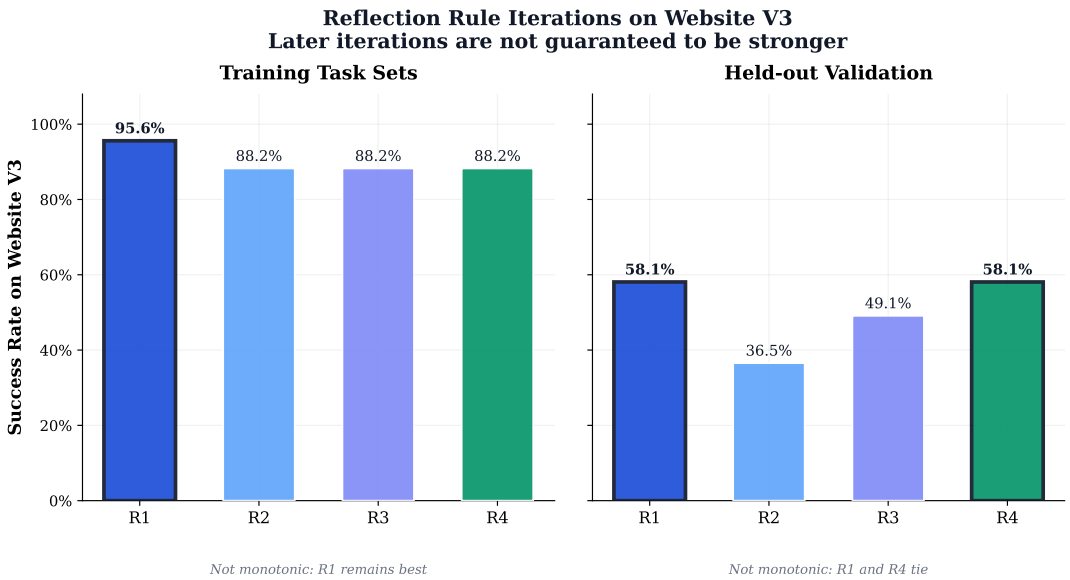

Figure 8: Aggregate website V3 performance for successive reflection-rule sets. The non-monotonic pattern supports the paper's claim that reflection-rule updates require verification before promotion.

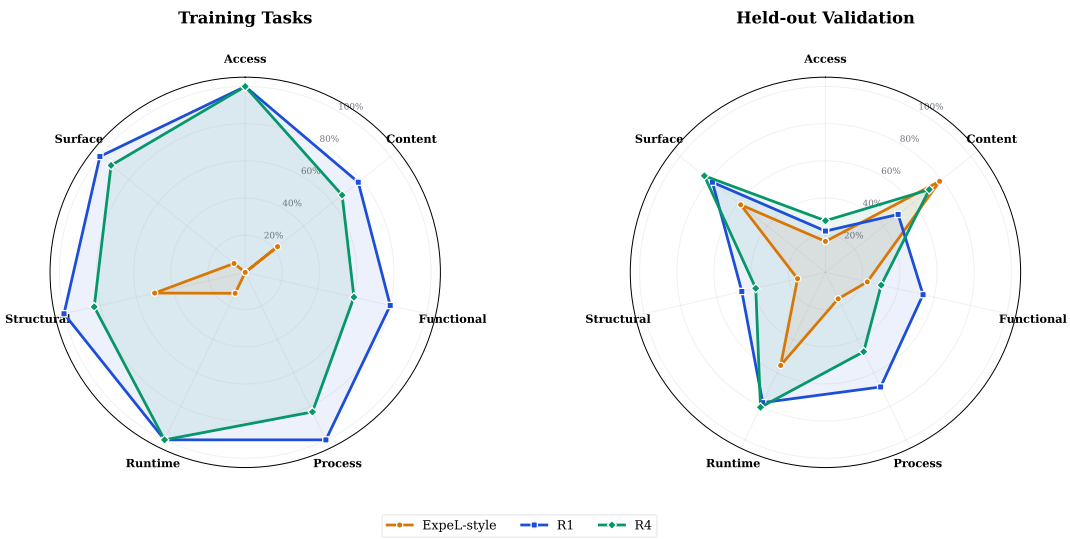

Figure 9: Radar summary of drift-family performance under the evaluated rule conditions. This shape-based view makes it easier to see whether a rule set improves robustness broadly or only along a few website-shift families.

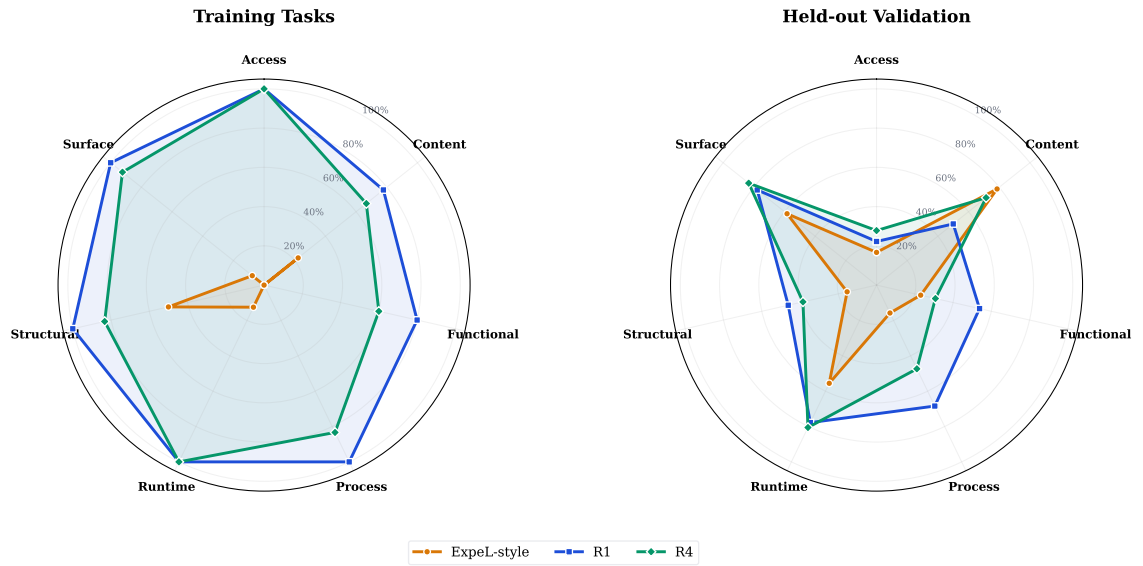

Figure 10: Per-drift website V3 success profiles for EXPEL-style rules and selected reflection-rule sets. R1 dominates the training tasks and ties the best held-out aggregate score, while R3 illustrates that a plausible later rule set can shift performance across drift families without improving the overall result.

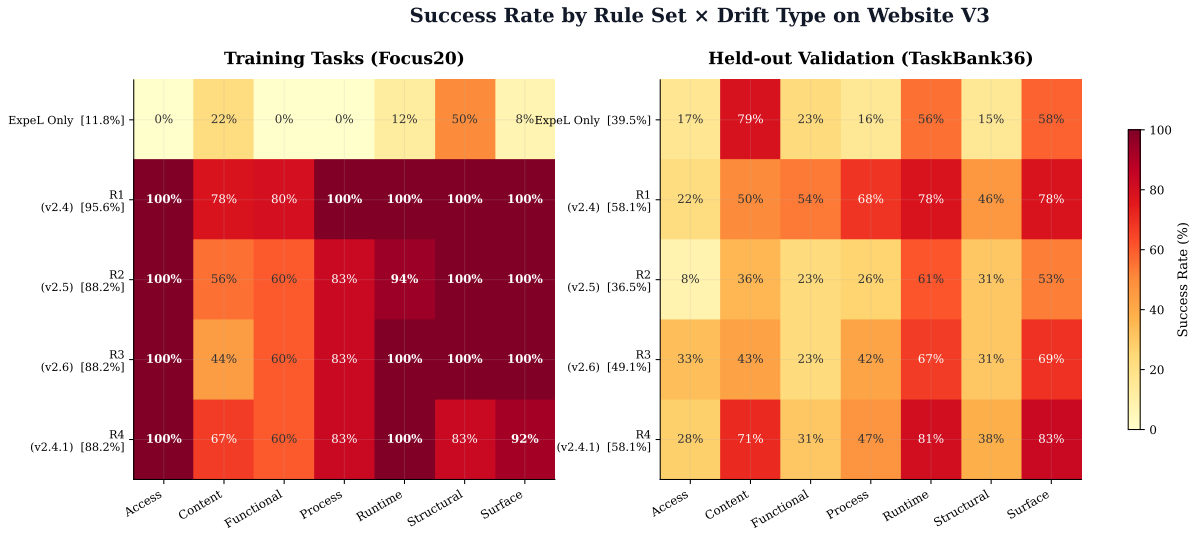

Figure 11: Full website V3 per-drift heatmap for EXPEL-style rules and all four reflection-rule sets R1–R4. This view complements the numeric tables by making drift-family regressions and improvements visually comparable across the training and held-out validation tasks.

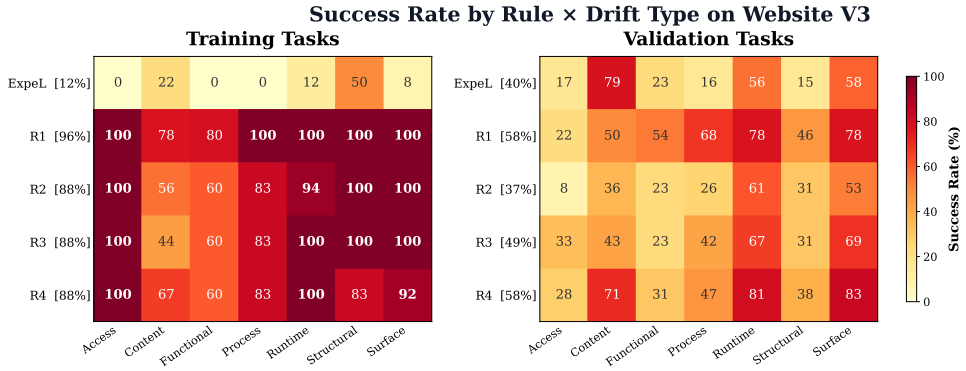

Figure 12: Poster-format website V3 heatmap. This rendering preserves the same R1–R4 ordering as the paper figures while using a wider layout for presentation material.

B.6 Claim Boundary Checklist

- Supported: EXPEL improves the training tasks on LINKDING V1.45.0 from $9/68$ to $56/68$.
- Supported: R1 improves over EXPEL-style rules on website V2 for both the training tasks and held-out validation tasks.
- Supported: R1 is strongest on the training tasks under website V3 at $65/68$.
- Supported: R1 and R4 tie on the held-out validation tasks under website V3 at $97/167$.
- Not claimed: later rulebooks always improve over earlier rulebooks.
- Not claimed: LINKDING single-site evidence establishes broad web-agent robustness.
- Intended next step: preserve R1's robust process/functional behavior while selectively importing R4's access/content/finalization gains.